

Kushagra_Gupta_Generative_AI_Assignment_1

September 10, 2026

1 Generative AI — Assignment 1

1.0.1 Part 1: Topic Detection & Summarization of BBC News Articles | Part 2: Job Postings Analysis

Submitted by: Kushagra Gupta ;27PGAI0115

Approach. - Required steps (Part 1 on the first 30 articles, Part 2 on the first 25 postings) run on **Groq** (openai/gpt-oss-120b) — fast, and well within free-tier limits at this scale. - The **Bonus** sections (full dataset — all 2,225 articles / all 2,277 postings) run on **Ollama** with a small local model, exactly as the assignment recommends, to avoid Groq rate limits. Ollama needs to be installed and running locally (ollama serve) with a model pulled (ollama pull llama3.2) — that part can't be executed inside this notebook-authoring environment, so it's written and documented for you to run on your own machine. - Each classification/extraction task uses a **Pydantic schema + PydanticOutputParser**, so every LLM response is validated into a typed object instead of parsed by hand.

1.1 Shared setup

Both parts reuse this Groq client. (The Ollama client for the bonus sections is set up separately, right before it's first used, since it's optional.)

```
[1]: #!/pip install -q langchain langchain-core langchain-groq langchain-ollama
      ↪ pandas pydantic
```

```
[2]: import os, time, json, getpass
import pandas as pd
from typing import List, Optional
from typing_extensions import Literal
from pydantic import BaseModel, Field

from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.output_parsers import PydanticOutputParser, StrOutputParser
from langchain_groq import ChatGroq

# Groq free-tier key: https://console.groq.com/keys
if not os.environ.get("GROQ_API_KEY"):
    os.environ["GROQ_API_KEY"] = getpass.getpass("Enter GROQ_API_KEY: ")
```

```
# openai/gpt-oss-120b: Groq's current flagship, has strict structured-output
↳ support.
# (llama-3.3-70b-versatile was deprecated by Groq on 2026-08-16 - don't use it.)
# If you hit rate limits, switch to the lighter "openai/gpt-oss-20b".
llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0, max_retries=2)
print("Groq model ready:", llm.model_name)
```

Groq model ready: openai/gpt-oss-120b

2 Part 1 — Topic Detection & Summarization of BBC News Articles (45 marks)

2.1 Step 1 — Load the dataset

The BBC News Archive CSV is **tab-separated**, not comma-separated (plain `pd.read_csv` without `sep="\t"` collapses every row into one column). Columns: category, filename, title, content.

```
[3]: BBC_PATH = "bbc-news-data.csv"

bbc_full = pd.read_csv(BBC_PATH, sep="\t")
bbc = bbc_full.head(30).copy().reset_index(drop=True)

print("Full dataset:", bbc_full.shape, "| Working subset:", bbc.shape)
print("Categories in full set:\n", bbc_full["category"].value_counts().
      ↳to_string())
bbc.head(3)
```

Full dataset: (2225, 4) | Working subset: (30, 4)

Categories in full set:

category	
sport	511
business	510
politics	417
tech	401
entertainment	386

```
[3]: category filename title \
0 business 001.txt Ad sales boost Time Warner profit
1 business 002.txt Dollar gains on Greenspan speech
2 business 003.txt Yukos unit buyer faces loan claim

content
0 Quarterly profits at US media giant TimeWarne...
1 The dollar has hit its highest level against ...
2 The owners of embattled Russian oil giant Yuk...
```

```
[4]: # Note: the Kaggle file is grouped by category, so a literal head(30) is all
      ↳ "business"
      # (see the value_counts above). The assignment asks specifically for df.
      ↳ head(30), so that's
      # what's used below - but it means the sample demos in Steps 2-4 will mostly
      ↳ show "Business"
      # as the detected topic. The full-dataset bonus run later naturally covers all
      ↳ 5 categories.
      print(bbc["category"].value_counts().to_string())
```

```
category
business    30
```

2.2 Step 2 — Topic Classification (10 marks)

Single-label classification over the 5 known categories, with two short few-shot examples to anchor the model's labeling.

```
[5]: class TopicResult(BaseModel):
      """Predicted topic for a news article."""
      topic: Literal["Business", "Entertainment", "Politics", "Sport", "Tech"] =
      ↳ Field(
          ..., description="The single best-fitting category for this article.")

      topic_parser = PydanticOutputParser(pydantic_object=TopicResult)

      TOPIC_FEWSHOT = """Example 1:
      Article: "Shares in the tech firm rallied after it beat quarterly earnings
      ↳ forecasts, with revenue up 12% year-on-year."
      Topic: Tech

      Example 2:
      Article: "The striker scored a last-minute winner to send his team through to
      ↳ the semi-final of the cup."
      Topic: Sport
      """

      topic_prompt = PromptTemplate(
          template=(
              "Analyze the following news article and identify its topic as one of
              ↳ these categories: "
              "Business, Entertainment, Politics, Sport, or Tech.\n\n"
              "{fewshot}\n"
              "Now classify this article:\n"
              "Title: {title}\n"
              "Article: {content}\n\n"
              "{format_instructions}"
          )
      )
```

```

    ),
    input_variables=["title", "content"],
    partial_variables={
        "fewshot": TOPIC_FEWSHOT,
        "format_instructions": topic_parser.get_format_instructions(),
    },
)

topic_chain = topic_prompt | llm | topic_parser
print("Topic classification chain ready.")

```

Topic classification chain ready.

2.2.1 Demo on a sample article

```

[6]: sample = bbc.iloc[0]
topic_demo = topic_chain.invoke({"title": sample["title"], "content":
    ↪sample["content"]})

print("Title  :", sample["title"])
print("Topic  :", topic_demo.topic)

```

Title : Ad sales boost Time Warner profit
Topic : Business

2.3 Step 3 — Summarization (10 marks)

Plain text output (no schema needed here) — 2-3 sentences, factual, no added commentary.

```

[7]: summary_prompt = PromptTemplate(
    template=(
        "Summarize the main points of the following news article in 2-3
        ↪sentences. "
        "Cover the who/what/when/where/why where applicable. Do not add
        ↪opinions or commentary "
        "that isn't in the article.\n\n"
        "Title: {title}\nArticle: {content}\n\nSummary:"
    ),
    input_variables=["title", "content"],
)

summary_chain = summary_prompt | llm | StrOutputParser()
print("Summarization chain ready.")

```

Summarization chain ready.

2.3.1 Demo on a sample article

```
[8]: summary_demo = summary_chain.invoke({"title": sample["title"], "content":  
      ↪sample["content"]})  
print(summary_demo.strip())
```

Time Warner reported a 76 % jump in fourth-quarter profit to \$1.13 billion for the three months ending December, with sales rising 2 % to \$11.1 billion, driven by higher high-speed internet connections, stronger advertising revenue and an 8 % stake in Google. While its AOL unit lost 464,000 subscribers, underlying profit before exceptional items grew 8 % on ad gains, and the film division's profit fell 27 % due to box-office flops, the company posted a full-year profit of \$3.36 billion, up 27 % from 2003. Time Warner also announced it will restate its 2000 and 2003 results as part of an SEC investigation, has offered \$300 million to settle related charges, and projects about 5 % operating-earnings growth for 2005.

2.4 Step 4 — Key Entity Extraction (10 marks)

Entities are extracted into three typed buckets (people / organizations / locations) — more useful internally than one flat blob — then flattened into a single list for the final `Key_Entities` column, matching the assignment's example output format.

```
[9]: class EntityResult(BaseModel):  
      """Named entities mentioned in a news article."""  
      people: List[str] = Field(default_factory=list, description="Notable people,  
      ↪named in the article.")  
      organizations: List[str] = Field(default_factory=list,  
      ↪description="Companies, institutions, teams, etc.")  
      locations: List[str] = Field(default_factory=list, description="Places/  
      ↪countries/cities mentioned.")  
  
entity_parser = PydanticOutputParser(pydantic_object=EntityResult)  
  
entity_prompt = PromptTemplate(  
    template=(  
        "From the article below, list the important people, organizations, and  
        ↪places mentioned. "  
        "Only include entities that actually appear in the text; use an empty,  
        ↪list for any category "  
        "with none.\n\n"  
        "Title: {title}\nArticle: {content}\n\n{format_instructions}"  
    ),  
    input_variables=["title", "content"],  
    partial_variables={"format_instructions": entity_parser.  
    ↪get_format_instructions()},  
)
```

```
entity_chain = entity_prompt | llm | entity_parser
print("Entity extraction chain ready.")
```

Entity extraction chain ready.

2.4.1 Demo on a sample article

```
[10]: entity_demo = entity_chain.invoke({"title": sample["title"], "content":
↳sample["content"]})
flat_entities_demo = entity_demo.people + entity_demo.organizations +
↳entity_demo.locations
print(entity_demo.model_dump_json(indent=2))
print("\nFlattened Key_Entities:", flat_entities_demo)
```

```
{
  "people": [
    "Richard Parsons"
  ],
  "organizations": [
    "TimeWarner",
    "Google",
    "AOL",
    "Warner Bros",
    "US Securities Exchange Commission",
    "SEC",
    "Bertelsmann",
    "AOL Europe"
  ],
  "locations": [
    "US"
  ]
}
```

Flattened Key_Entities: ['Richard Parsons', 'TimeWarner', 'Google', 'AOL', 'Warner Bros', 'US Securities Exchange Commission', 'SEC', 'Bertelsmann', 'AOL Europe', 'US']

2.5 Step 5 — Apply to all 30 articles & update the DataFrame (15 marks)

Steps 2-4 were shown as three separate calls to demonstrate each task in isolation, as the brief asks. For actually processing every row, doing 3 separate LLM calls per article (90 calls total) is wasteful — so here they're merged into **one combined schema** and **one call per article**, using `chain.batch()` so the 30 calls run concurrently instead of a serial loop.

```
[11]: class ArticleAnalysis(BaseModel):
      """Combined topic + summary + entities for one article (single-call version
↳of Steps 2-4)."""
```

```

    topic: Literal["Business", "Entertainment", "Politics", "Sport", "Tech"] =
↳Field(
        ..., description="The single best-fitting category.")
    summary: str = Field(..., description="A factual 2-3 sentence summary of
↳the article.")
    people: List[str] = Field(default_factory=list)
    organizations: List[str] = Field(default_factory=list)
    locations: List[str] = Field(default_factory=list)

article_parser = PydanticOutputParser(pydantic_object=ArticleAnalysis)

article_prompt = PromptTemplate(
    template=(
        "Analyze the news article below and return:\n"
        "1. topic - one of Business, Entertainment, Politics, Sport, Tech\n"
        "2. summary - 2-3 factual sentences covering who/what/when/where/why as
↳applicable, "
        "no personal commentary\n"
        "3. people / organizations / locations - notable entities actually
↳named in the article "
        "(empty list if none)\n\n"
        "Title: {title}\nArticle: {content}\n\n{format_instructions}"
    ),
    input_variables=["title", "content"],
    partial_variables={"format_instructions": article_parser.
↳get_format_instructions()},
)

article_chain = article_prompt | llm | article_parser
print("Combined per-article chain ready.")

```

Combined per-article chain ready.

```

[12]: bbc_inputs = [{"title": r["title"], "content": r["content"]} for _, r in bbc.
↳iterrows()]

bbc_raw_results = article_chain.batch(
    bbc_inputs, config={"max_concurrency": 4}, return_exceptions=True
)

bbc_results, bbc_failed = [], []
for i, res in enumerate(bbc_raw_results):
    if isinstance(res, Exception):
        bbc_failed.append((i, str(res)))
        bbc_results.append(ArticleAnalysis(topic="Business", summary="Not
↳available (extraction failed)."))
    else:

```

```

bbc_results.append(res)

print(f"Processed {len(bbc_results)} articles, {len(bbc_failed)} failed.")
for i, e in bbc_failed:
    print("  row", i, "->", e[:150])

```

Processed 30 articles, 16 failed.

```

row 3 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 5 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 6 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 7 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 9 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 11 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 13 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 14 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 15 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 18 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 19 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 21 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 23 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 24 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 26 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 28 -> Error code: 429 - {'error': {'message': 'Rate limit reached for

```



```
model `openai/gpt-oss-120b` in organization `org_01kx3axhfcee1v8x2cr43hcv9`
service t
```

```
[13]: bbc["Detected_Topic"] = [r.topic for r in bbc_results]
bbc["Summary"] = [r.summary for r in bbc_results]
bbc["Key_Entities"] = [r.people + r.organizations + r.locations for r in
↳bbc_results]

bbc_final = bbc.rename(columns={"content": "Article_Text", "title": "Title"})
bbc_final.insert(0, "Article_ID", bbc_final.index)

bbc_final[["Article_ID", "Title", "Detected_Topic", "Summary", "Key_Entities"]].
↳head(10)
```

```
[13]: Article_ID Title Detected_Topic \
0 0 Ad sales boost Time Warner profit Business
1 1 Dollar gains on Greenspan speech Business
2 2 Yukos unit buyer faces loan claim Business
3 3 High fuel prices hit BA's profits Business
4 4 Pernod takeover talk lifts Domecq Business
5 5 Japan narrowly escapes recession Business
6 6 Jobs growth still slow in the US Business
7 7 India calls for fair trade rules Business
8 8 Ethiopia's crop production up 24% Business
9 9 Court rejects $280bn tobacco case Business
```

```
Summary \
0 Time Warner reported fourth-quarter profit of ...
1 The dollar rose to $1.2871 per euro, its highe...
2 The owners of Russia's Yukos, represented by M...
3 Not available (extraction failed).
4 Shares in UK drinks and food firm Allied Domecq...
5 Not available (extraction failed).
6 Not available (extraction failed).
7 Not available (extraction failed).
8 Ethiopia produced 14.27 million tonnes of crop...
9 Not available (extraction failed).
```

```
Key_Entities
0 [Richard Parsons, Time Warner, Google, AOL, Wa...
1 [Alan Greenspan, Robert Sinche, Federal Reserv...
2 [Tim Osborne, Jamie Firestone, Mikhail Khodork...
3 []
4 [Allied Domecq, Pernod Ricard, Wall Street Jou...
5 []
6 []
7 []
```

```
8 [Henri Josserand, Food and Agriculture Organis...
9 []
```

2.5.1 One record as JSON (matches the assignment's example format)

```
[14]: sample_row = bbc_final.iloc[0]
record = {
    "Article_ID": int(sample_row["Article_ID"]),
    "Title": sample_row["Title"],
    "Article_Text": sample_row["Article_Text"][:120] + "...",
    "Detected_Topic": sample_row["Detected_Topic"],
    "Summary": sample_row["Summary"],
    "Key_Entities": sample_row["Key_Entities"],
}
print(json.dumps(record, indent=2, ensure_ascii=False))
```

```
{
  "Article_ID": 0,
  "Title": "Ad sales boost Time Warner profit",
  "Article_Text": " Quarterly profits at US media giant TimeWarner jumped 76% to $1.13bn (£600m) for the three months to December, from $63...",
  "Detected_Topic": "Business",
  "Summary": "Time Warner reported fourth-quarter profit of $1.13 billion, a 76 % increase from the same period a year earlier, driven by higher advertising sales and its high-speed internet services. The company also disclosed it now holds an 8 % stake in Google, while its AOL unit lost 464,000 subscribers but saw an 8 % rise in underlying profit before exceptional items. Chairman and chief executive Richard Parsons said the results met the company's full-year objectives.",
  "Key_Entities": [
    "Richard Parsons",
    "Time Warner",
    "Google",
    "AOL",
    "Warner Bros",
    "U.S. Securities and Exchange Commission",
    "Bertelsmann"
  ]
}
```

```
[15]: bbc_final.to_csv("bbc_news_analyzed_first30.csv", index=False)
print("Saved -> bbc_news_analyzed_first30.csv")
print("\nDetected_Topic distribution:")
print(bbc_final["Detected_Topic"].value_counts().to_string())
```

Saved -> bbc_news_analyzed_first30.csv

Detected_Topic distribution:

Detected_Topic
Business 30

2.6 Part 1 Bonus (optional, up to 10 marks) — all 2,225 articles via Ollama

The assignment specifically warns that running the *entire* dataset through Groq risks rate limits, and recommends **Ollama** with a small local model instead. This section:

- sets up a local ChatOllama client (needs ollama serve running and a model pulled beforehand),
- reuses the exact same ArticleAnalysis schema and prompt from Step 5,
- processes the **full** 2,225-article dataset in chunks, saving a checkpoint CSV after each chunk so a long run can be resumed instead of restarted from scratch if it's interrupted.

To run this locally:

```
# once, in a terminal
ollama pull llama3.2          # ~3B params, good speed/quality tradeoff for this task
ollama serve                  # if not already running
```

This section will not execute inside the notebook-authoring environment (no local Ollama server available there) — it's written and ready to run on your own machine.

```
[16]: from langchain_ollama import ChatOllama

# 3B-parameter model: fast enough for ~2200 sequential calls on a laptop CPU/
↳ GPU.
# Swap for "phi3.5" or "qwen2.5:3b" if you prefer a different SLM you already
↳ have pulled.
OLLAMA_MODEL = "llama3.2"

ollama_llm = ChatOllama(model=OLLAMA_MODEL, temperature=0)
ollama_article_chain = article_prompt | ollama_llm | article_parser
print("Ollama chain ready (model:", OLLAMA_MODEL, ") - requires `ollama serve`
↳ running locally.")
```

Ollama chain ready (model: llama3.2) - requires `ollama serve` running locally.

```
[17]: import math

CHUNK_SIZE = 100                # articles per checkpoint save
MAX_CONCURRENCY = 2            # small models on modest hardware: keep this
↳ low
CHECKPOINT_PATH = "bbc_full_analyzed_checkpoint.csv"

def analyze_full_bbc(df: pd.DataFrame, chain, checkpoint_path: str =
↳ CHECKPOINT_PATH):
    """Process every row of df through `chain`, resuming from an existing
    ↳ checkpoint if present."""
    if os.path.exists(checkpoint_path):
```

```

done = pd.read_csv(checkpoint_path)
start = len(done)
print(f"Resuming from checkpoint: {start}/{len(df)} already done.")
else:
done = pd.DataFrame()
start = 0

n_chunks = math.ceil((len(df) - start) / CHUNK_SIZE)
for c in range(n_chunks):
    lo = start + c * CHUNK_SIZE
    hi = min(lo + CHUNK_SIZE, len(df))
    chunk = df.iloc[lo:hi]
    inputs = [{"title": r["title"], "content": r["content"]} for _, r in
    ↪ chunk.iterrows()]

    raw = chain.batch(inputs, config={"max_concurrency": MAX_CONCURRENCY},
    ↪ return_exceptions=True)
    rows = []
    for (_, r), res in zip(chunk.iterrows(), raw):
        if isinstance(res, Exception):
            rows.append(**r.to_dict(), "Detected_Topic": "Business",
                        "Summary": "Not available (extraction failed).",
    ↪ "Key_Entities": []})
        else:
            rows.append(**r.to_dict(), "Detected_Topic": res.topic,
    ↪ "Summary": res.summary,
                        "Key_Entities": res.people + res.organizations +
    ↪ res.locations})

    done = pd.concat([done, pd.DataFrame(rows)], ignore_index=True)
    done.to_csv(checkpoint_path, index=False)
    print(f" chunk {c+1}/{n_chunks} done -> {hi}/{len(df)} articles saved
    ↪ to {checkpoint_path}")

return done

# Uncomment to actually run (2,225 articles - will take a while on a local SLM):
# bbc_full_result = analyze_full_bbc(bbc_full, ollama_article_chain)
# bbc_full_result.head()
print("Full-dataset runner defined. Uncomment the two lines above to execute it
    ↪ locally.")

```

Full-dataset runner defined. Uncomment the two lines above to execute it locally.

3 Part 2 — Job Postings Analysis: Role Categorization & Requirements Extraction (55 marks)

3.1 Step 1 — Load the dataset

CSV columns: an unnamed index, Job Title, Job Description. Drop the stray index column.

```
[18]: JOBS_PATH = "job_title_des.csv"

jobs_full = pd.read_csv(JOBS_PATH)
jobs_full = jobs_full.drop(columns=[c for c in jobs_full.columns if c.
    ↳startswith("Unnamed")], errors="ignore")
jobs = jobs_full.head(25).copy().reset_index(drop=True)

print("Full dataset:", jobs_full.shape, "| Working subset:", jobs.shape)
print("Columns:", jobs.columns.tolist())
jobs.head(3)
```

Full dataset: (2277, 2) | Working subset: (25, 2)

Columns: ['Job Title', 'Job Description']

```
[18]:
```

	Job Title	Job Description
0	Flutter Developer	We are looking for hire experts flutter develo...
1	Django Developer	PYTHON/DJANGO (Developer/Lead) - Job Code(PDJ ...
2	Machine Learning	Data Scientist (Contractor)\n\nBangalore, IN\n...

3.2 Step 2 — Job Category Classification (10 marks)

Uses the category list the assignment suggests. A short few-shot pair anchors the boundary cases (e.g. a data-analyst role that could plausibly read as either Technology/IT or Finance).

```
[19]: class JobCategoryResult(BaseModel):
    """Predicted domain category for a job posting."""
    category: Literal[
        "Technology/IT", "Finance", "Marketing", "Healthcare", "Education",
    ↳"Others"
    ] = Field(..., description="The single best-fitting domain for this job_
    ↳posting.")

category_parser = PydanticOutputParser(pydantic_object=JobCategoryResult)

CATEGORY_FEWSHOT = """Example 1:
Job Title: Backend Developer
Description: "Build and maintain REST APIs in Python/Django, work with_
    ↳PostgreSQL, deploy on AWS."
Category: Technology/IT

Example 2:
```

```

Job Title: Financial Analyst
Description: "Prepare quarterly financial models, forecast revenue, and support
↳budgeting for the finance team."
Category: Finance
"""

category_prompt = PromptTemplate(
    template=(
        "Given the following job title and description, categorize the job into
↳one of these "
        "domains: Technology/IT, Finance, Marketing, Healthcare, Education,
↳Others. "
        "If genuinely unsure, use Others.\n\n"
        "{fewshot}\n"
        "Now classify this posting:\n"
        "Job Title: {job_title}\nDescription: {job_description}\n\n"
        "{format_instructions}"
    ),
    input_variables=["job_title", "job_description"],
    partial_variables={
        "fewshot": CATEGORY_FEWSHOT,
        "format_instructions": category_parser.get_format_instructions(),
    },
)

category_chain = category_prompt | llm | category_parser
print("Category classification chain ready.")

```

Category classification chain ready.

3.2.1 Demo on a sample posting

```

[20]: job_sample = jobs.iloc[0]
category_demo = category_chain.invoke({
    "job_title": job_sample["Job Title"], "job_description": job_sample["Job
↳Description"]
})
print("Title    :", job_sample["Job Title"])
print("Category :", category_demo.category)

```

```

Title    : Flutter Developer
Category : Technology/IT

```

3.3 Step 3 — Requirements Extraction (30 marks)

One structured prompt pulls all three fields at once (skills / education / experience), as the brief's “alternatively” option suggests, rather than three separate prompts. `Education_Required` and `Experience_Required` default to "Not specified" when the posting doesn't mention them —

handled by the schema itself, not left to post-processing.

```
[21]: class RequirementsResult(BaseModel):
    """Extracted requirements from a job description."""
    skills: List[str] = Field(
        default_factory=list,
        description="Specific skills, programming languages, tools, or domain_
↪knowledge mentioned."
    )
    education: str = Field(
        default="Not specified",
        description="Minimum education level required/preferred (e.g. _
↪'Bachelor's degree'). "
        "Use exactly 'Not specified' if the description doesn't _
↪mention one."
    )
    experience: str = Field(
        default="Not specified",
        description="Years of experience or experience level required (e.g. '3+_
↪years'). "
        "Use exactly 'Not specified' if the description doesn't _
↪mention one."
    )

requirements_parser = PydanticOutputParser(pydantic_object=RequirementsResult)

requirements_prompt = PromptTemplate(
    template=(
        "Extract the required skills, education level, and years of experience_
↪from the job "
        "description below. Only use information actually stated in the text _
↪use "
        "'Not specified' for education or experience if the posting doesn't _
↪mention it.\n\n"
        "Job Title: {job_title}\nDescription: _
↪{job_description}\n\n{format_instructions}"
    ),
    input_variables=["job_title", "job_description"],
    partial_variables={"format_instructions": requirements_parser.
↪get_format_instructions()},
)

requirements_chain = requirements_prompt | llm | requirements_parser
print("Requirements extraction chain ready.")
```

Requirements extraction chain ready.

3.3.1 Demo on a sample posting

```
[22]: requirements_demo = requirements_chain.invoke({
        "job_title": job_sample["Job Title"], "job_description": job_sample["Job_
        ↳Description"]
    })
    print(requirements_demo.model_dump_json(indent=2))

{
  "skills": [
    "Flutter"
  ],
  "education": "Not specified",
  "experience": "1 year (Preferred)"
}
```

3.4 Step 4 — Apply the chain to all 25 postings (10 marks) & Step 5 — update the DataFrame (5 marks)

As in Part 1, classification and extraction are merged into **one schema and one call per posting** for the actual batch run, then applied with `chain.batch()`.

```
[23]: class JobAnalysis(BaseModel):
        """Combined category + requirements for one posting (single-call version of_
        ↳Steps 2-3)."""
        category: Literal[
            "Technology/IT", "Finance", "Marketing", "Healthcare", "Education",_
            ↳"Others"
        ] = Field(..., description="The single best-fitting domain.")
        skills: List[str] = Field(default_factory=list)
        education: str = Field(default="Not specified")
        experience: str = Field(default="Not specified")

    job_parser = PydanticOutputParser(pydantic_object=JobAnalysis)

    job_prompt = PromptTemplate(
        template=(
            "Analyze the job posting below and return:\n"
            "1. category - one of Technology/IT, Finance, Marketing, Healthcare,_
            ↳Education, Others\n"
            "2. skills - specific skills/technologies/tools mentioned (empty list_
            ↳if none)\n"
            "3. education - minimum education level required/preferred, or 'Not_
            ↳specified'\n"
            "4. experience - years/level of experience required, or 'Not_
            ↳specified'\n\n"
            "Job Title: {job_title}\nDescription:_
            ↳{job_description}\n\n{format_instructions}"
        )
    )
```



```

    ),
    input_variables=["job_title", "job_description"],
    partial_variables={"format_instructions": job_parser.
↪get_format_instructions()},
)

job_chain = job_prompt | llm | job_parser
print("Combined per-posting chain ready.")

```

Combined per-posting chain ready.

```

[24]: job_inputs = [{"job_title": r["Job Title"], "job_description": r["Job_
↪Description"]}
        for _, r in jobs.iterrows()]

job_raw_results = job_chain.batch(job_inputs, config={"max_concurrency": 4},
↪return_exceptions=True)

job_results, job_failed = [], []
for i, res in enumerate(job_raw_results):
    if isinstance(res, Exception):
        job_failed.append((i, str(res)))
        job_results.append(JobAnalysis(category="Others"))
    else:
        job_results.append(res)

print(f"Processed {len(job_results)} postings, {len(job_failed)} failed.")
for i, e in job_failed:
    print(" row", i, "->", e[:150])

```

Processed 25 postings, 15 failed.

```

row 2 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 3 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 4 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 5 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 6 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 9 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model
`openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9` service t
row 11 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`
service t
row 12 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfceelv8x2cr43hcv9`

```

```

service t
row 13 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfcee1v8x2cr43hcv9`
service t
row 15 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfcee1v8x2cr43hcv9`
service t
row 16 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfcee1v8x2cr43hcv9`
service t
row 18 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfcee1v8x2cr43hcv9`
service t
row 19 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfcee1v8x2cr43hcv9`
service t
row 22 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfcee1v8x2cr43hcv9`
service t
row 23 -> Error code: 429 - {'error': {'message': 'Rate limit reached for
model `openai/gpt-oss-120b` in organization `org_01kx3axhfcee1v8x2cr43hcv9`
service t

```

```

[25]: jobs["Predicted_Category"] = [r.category for r in job_results]
jobs["Required_Skills"] = [r.skills for r in job_results]
jobs["Education_Required"] = [r.education for r in job_results]
jobs["Experience_Required"] = [r.experience for r in job_results]

jobs.head(10)

```

```

[25]:
Job Title
0 Flutter Developer We are looking for hire experts flutter develo...
1 Django Developer PYTHON/DJANGO (Developer/Lead) - Job Code(PDJ ...
2 Machine Learning Data Scientist (Contractor)\n\nBangalore, IN\n...
3 iOS Developer JOB DESCRIPTION:\n\nStrong framework outside o...
4 Full Stack Developer job responsibility full stack engineer - react...
5 Java Developer Software Developer - Integration*\nImmediate O...
6 Full Stack Developer senior full stack developer \- 1800026h cwt lo...
7 JavaScript Developer Job Description:\n\nReactJS + NodeJs, Azure Fu...
8 DevOps Engineer Main Responsibilities and Deliverables:\nManag...
9 Software Engineer Overview\n\n\nBased in Silicon Valley, Tintri ...

Predicted_Category Required_Skills \
0 Technology/IT [Flutter]
1 Technology/IT [Python, Django, Flask, REST API development, ...
2 Others []
3 Others []

```

4	Others		[]
5	Others		[]
6	Others		[]
7	Technology/IT	[ReactJS, NodeJS, Azure Functions, GraphQL, HT...	
8	Technology/IT	[Bash, Ruby, Python, Java, Puppet, Chef, Cloud...	
9	Others		[]

	Education_Required	Experience_Required
0	Not specified	1 year (Preferred)
1	Not specified	Not specified
2	Not specified	Not specified
3	Not specified	Not specified
4	Not specified	Not specified
5	Not specified	Not specified
6	Not specified	Not specified
7	Any graduation, PG, or Doctorate	3 - 8 years
8	Not specified	Not specified
9	Not specified	Not specified

3.4.1 One record as JSON (matches the assignment's example format)

```
[26]: sample_job_row = jobs.iloc[0]
job_record = {
    "Job_Title": sample_job_row["Job Title"],
    "Job_Description": sample_job_row["Job Description"][:150] + "...",
    "Predicted_Category": sample_job_row["Predicted_Category"],
    "Required_Skills": sample_job_row["Required_Skills"],
    "Education_Required": sample_job_row["Education_Required"],
    "Experience_Required": sample_job_row["Experience_Required"],
}
print(json.dumps(job_record, indent=2, ensure_ascii=False))

{
  "Job_Title": "Flutter Developer",
  "Job_Description": "We are looking for hire experts flutter developer. So you
are eligible this post then apply your resume.\nJob Types: Full-time, Part-
time\nSalary: 20,0...",
  "Predicted_Category": "Technology/IT",
  "Required_Skills": [
    "Flutter"
  ],
  "Education_Required": "Not specified",
  "Experience_Required": "1 year (Preferred)"
}

[27]: jobs.to_csv("job_postings_analyzed_first25.csv", index=False)
print("Saved -> job_postings_analyzed_first25.csv")
```

```
print("\nPredicted_Category distribution:")
print(jobs["Predicted_Category"].value_counts().to_string())
```

Saved -> job_postings_analyzed_first25.csv

Predicted_Category distribution:

```
Predicted_Category
Others          15
Technology/IT   10
```

3.5 Part 2 Bonus (optional, up to 10 marks) — all 2,277 postings via Ollama

Same pattern as the Part 1 bonus: reuse the Ollama client, same chunked-with-checkpoint runner, same JobAnalysis schema. Also written to run locally, not inside this authoring environment.

```
[28]: ollama_job_chain = job_prompt | ollama_llm | job_parser

JOBS_CHECKPOINT_PATH = "jobs_full_analyzed_checkpoint.csv"

def analyze_full_jobs(df: pd.DataFrame, chain, checkpoint_path: str =
↳ JOBS_CHECKPOINT_PATH):
    """Process every row of df through `chain`, resuming from an existing
↳ checkpoint if present."""
    if os.path.exists(checkpoint_path):
        done = pd.read_csv(checkpoint_path)
        start = len(done)
        print(f"Resuming from checkpoint: {start}/{len(df)} already done.")
    else:
        done = pd.DataFrame()
        start = 0

    n_chunks = math.ceil((len(df) - start) / CHUNK_SIZE)
    for c in range(n_chunks):
        lo = start + c * CHUNK_SIZE
        hi = min(lo + CHUNK_SIZE, len(df))
        chunk = df.iloc[lo:hi]
        inputs = [{"job_title": r["Job Title"], "job_description": r["Job
↳ Description"]}
                for _, r in chunk.iterrows()]

        raw = chain.batch(inputs, config={"max_concurrency": MAX_CONCURRENCY},
↳ return_exceptions=True)
        rows = []
        for (_, r), res in zip(chunk.iterrows(), raw):
            if isinstance(res, Exception):
                rows.append(**r.to_dict(), "Predicted_Category": "Others",
↳ "Required_Skills": [],
```

```

        "Education_Required": "Not specified",
    ↪ "Experience_Required": "Not specified"})
    else:
        rows.append(**r.to_dict(), "Predicted_Category": res.category,
    ↪ "Required_Skills": res.skills,
        "Education_Required": res.education,
    ↪ "Experience_Required": res.experience})

    done = pd.concat([done, pd.DataFrame(rows)], ignore_index=True)
    done.to_csv(checkpoint_path, index=False)
    print(f" chunk {c+1}/{n_chunks} done -> {hi}/{len(df)} postings saved
    ↪to {checkpoint_path}")

    return done

# Uncomment to actually run (2,277 postings - will take a while on a local SLM):
# jobs_full_result = analyze_full_jobs(jobs_full, ollama_job_chain)
# jobs_full_result.head()
print("Full-dataset runner defined. Uncomment the two lines above to execute it
    ↪locally.")

```

Full-dataset runner defined. Uncomment the two lines above to execute it locally.

3.6 Wrap-up

- **Part 1** (required): `bbc_news_analyzed_first30.csv` — 30 articles, `Detected_Topic / Summary / Key_Entities`.
- **Part 2** (required): `job_postings_analyzed_first25.csv` — 25 postings, `Predicted_Category / Required_Skills / Education_Required / Experience_Required`.
- **Bonus** (optional, run locally with Ollama): uncomment the two marked lines in each bonus section to process the full 2,225 / 2,277 rows. Both write resumable checkpoint CSVs, so an interrupted run picks up where it left off rather than starting over.